

NBA Foul Predictor

Imports

```
import numpy as np
import polars as pl
from pyprojroot.here import here

## To fix format of PDF output
pl.Config.set_tbl_cols(8)    ## show at most 8 columns (first 4 and last 4)
pl.Config.set_tbl_rows(6)    ## show at most 6 rows
```

```
polars.config.Config
```

Combining the Referee Stats Files Into One Table

```
seasons = ['2017-2018', '2018-2019', '2019-2020', '2020-2021', '2021-2022',
           '2022-2023', '2023-2024', '2024-2025', '2025-2026']

refList = []
for s in seasons:
    fileName = f'nba_referee_stats_{s}.csv'
    if s == '2025-2026':
        fileName = 'nba_referee_stats_2025_2026.csv' ## uses an underscore
        instead of dash between years

    refSeason = pl.read_csv(here(f'Data/{fileName}'))

    ## find foul differential column
    diffCol = [c for c in refSeason.columns if 'FOUL DIFFERENTIAL' in c][0]

    refSeason = (refSeason
                 .rename({diffCol: 'foul_diff'})
                 .rename({'TYPE': 'ROLE'}, strict = False) ## older files say TYPE
                 .drop(['RANK', 'Unnamed: 13'], strict = False) ## empty columns
                 .with_columns(Season = pl.lit(s)) ## adds Season if missing
                 )
    refList.append(refSeason)

## Stack all seasons
refStats = pl.concat(refList, how = 'diagonal')
```

```
## Renaming columns
refStats = refStats.rename({
  'Season': 'season', 'REFEREE': 'referee', 'ROLE': 'role', 'GENDER': 'gender',
  'EXPERIENCE (YEARS)': 'experience', 'GAMES OFFICIATED': 'games', 'HOME TEAM
WIN%': 'home_win_pct', 'HOME TEAM POINTS DIFFERENTIAL': 'home_pt_diff', 'TOTAL
POINTS PER GAME': 'points_per_game', 'CALLED FOULS PER GAME': 'fouls_per_game',
'FOUL% AGAINST ROAD TEAMS': 'foul_pct_road', 'FOUL% AGAINST HOME TEAMS':
'foul_pct_home',
})

## Fixing percents
refStats = refStats.with_columns(
  pl.when(pl.col(c) > 1).then(pl.col(c) / 100).otherwise(pl.col(c)).alias(c)
  for c in ['foul_pct_road', 'foul_pct_home']
)

refStats
```

season	referee	role	experience	...	foul_pct_road	foul_pct_home	foul_diff	gender
str	str	str	i64	...	f64	f64	f64	str
"2017-2018"	"Ben Taylor"	"MAIN"	6	...	0.538	0.462	3.0	null
"2017-2018"	"Bennie Adams"	"MAIN"	22	...	0.498	0.502	-0.1	null
"2017-2018"	"Bill Kennedy"	"MAIN"	22	...	0.51	0.49	0.8	null
...	...	...	...	...	...	...	...	...
"2025-2026"	"Eric Lewis"	"CHIEF"	22	...	0.539	0.461	3.2	"MALE"
"2025-2026"	"Angel Kent"	"CREW"	1	...	0.447	0.553	-4.0	"FE-MALE"
"2025-2026"	"Ben Taylor"	"CHIEF"	15	...	0.509	0.491	0.6	"MALE"

Combine CHEIF and CREW Rows

```
avgCols = ['home_win_pct', 'home_pt_diff', 'points_per_game', 'fouls_per_game',
'foul_pct_road', 'foul_pct_home', 'foul_diff']
```

```

refStats = (refStats
  .group_by(['season', 'referee']) ## referee rows for a season together
  .agg(
    experience = pl.col('experience').max(),
    gender = pl.col('gender').first(),
    games = pl.col('games').sum(), ## adding them
    *[(pl.col(c) * pl.col('games')).sum() / pl.col('games').sum()).alias(c)
  for c in avgCols] ## weighted average for every column in avgCols
  )
  .sort(['season', 'referee'])
)
refStats

```

season	referee	home_win_pct	home_pt_diff.	foul_diff	experience	gender	games	
str	str	f64	f64	...	f64	i64	str	i64
"2017-2018"	"Aaron Smith"	0.625	2.8	...	1.1	2	null	56
"2017-2018"	"Ben Taylor"	0.644823	5.606452	...	-0.73871	6	null	62
"2017-2018"	"Bennie Adams"	0.476615	0.5	...	0.176923	22	null	65
...	...	...	...	...	...	...	...	...
"2025-2026"	"Tyler Ford"	0.592797	4.215254	...	-0.452542	12	"MALE"	59
"2025-2026"	"Tyler Ricks"	0.55	-1.5	...	0.8	5	"MALE"	40
"2025-2026"	"Zach Zarba"	0.5667	1.526667	...	-0.826667	23	"MALE"	60

Cleaning Games File

```

games = pl.read_csv(here('Data/nba_games_ref_assignments.csv'))

games = (games
  ## Splitting season code (22025 --> game type 2, start year 2025)
  .with_columns(
    game_type = pl.col('season') // 10000,

```

```

        start_year = pl.col('season') % 10000,
    )
    ## Make it look like the referee table above ('2025-2026')
    .with_columns(
        season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year')
+ 1).cast(pl.String)
    )
    ## Keep only regular season games (2017-18)
    .filter(pl.col('game_type') == 2, pl.col('start_year') >= 2017)
    ## Fix team names
    .with_columns(
        pl.col(['home_team', 'away_team']).str.replace('L.A.', 'LA', literal=True)
    )
    ## Remove games listed twice
    .unique(subset = 'game_id', keep = 'first', maintain_order=True)
    ## Remove games missing referees
    .drop_nulls(subset = ['official1', 'official2', 'official3'])
    ## Drop 4th referee column (empty during reg season)
    .drop(['official4', 'official4_code', 'official4_JNum'])
)
games.shape

```

```
(10774, 21)
```

Matching Referee Stats to Games File - Fixing Names

```

## Referee Name fixed
nameFix = {
    'J.T. Orr': 'JT Orr',
    'Sharae Mitchell': "Sha'Rae Mitchell", 'ShaRae Mitchell': "Sha'Rae Mitchell",
    'Pat OConnell': "Pat O'Connell", 'PAT OConnell': "Pat O'Connell", "PAT
O'Connell": "Pat O'Connell",
    'Cheryl Flores': 'Che Flores',
    'Jamahl Ralls': 'JD Ralls',
    'CP Painter': 'Chelisa Painter',
}
refStats = refStats.with_columns(pl.col('referee').replace(nameFix))

## One row per referee per game, attaching stats
gameRefs = (games
    .unpivot(index = ['game_id', 'season'], on = ['official1', 'official2',
'official3'],
        variable_name = 'slot', value_name = 'referee')

```

```

.with_columns(pl.col('referee').replace(nameFix))
.join(refStats, on = ['season', 'referee'], how = 'left')
)
## Check - should be 0
gameRefs['fouls_per_game'].null_count()

```

```
0
```

Average 3 Referees and Attach Games

```

crewStats = (gameRefs
.group_by('game_id')
.agg(
    crew_fouls_per_game = pl.col('fouls_per_game').mean(),
    crew_experience = pl.col('experience').mean(),
    crew_foul_diff = pl.col('foul_diff').mean(),
    crew_home_win_pct = pl.col('home_win_pct').mean(),
)
)

## Attaching stats to games_TranslateTable
games = games.join(crewStats, on = 'game_id', how = 'left')
games

```

game_id	game_date	season	game_code...	crew_fouls_per_game	crew_experience	crew_foul_diff	crew_home_win_pct
i64	str	str	str ...	f64	f64	f64	f64
22501186	"04/12/2026"	2025-2026	"20260412/ ... ORL- BOS"	39.378543	14.666667	0.264307	0.617964
22501187	"04/12/2026"	2025-2026	"20260412/ ... WAS- CLE"	39.585743	18.666667	-0.231223	0.546168
22501188	"04/12/2026"	2025-2026	"20260412/ ... DETIND"	39.836612	12.666667	-0.028583	0.538216
...	...	...	...	...	...	...	...
21700013	"10/18/2017"	2017-2018	"20171018/ ... HOUSAC"	39.12652	19.0	0.492137	0.56759
21700001	"10/17/2017"	2017-2018	"20171017/ ... BOSCLE"	40.411111	20.0	0.607516	0.565694

game_id	game_date	season	game_code...	crew_fouls	per_game...	crew_foul_diff	home_win_pct
i64	str	str	str	...	f64	f64	f64
21700002	"10/17/2017"	2017-2018	"20171017/ ... HOUGSW"	39.970385	19.666667	0.627942	0.582281

Attaching Team Fouls File

```
teamFouls =
(pl.read_csv(here('Data/nba_team_personal_fouls_per_game_2017_2025.csv'))
 .with_columns(pl.col('team').replace({'Okla City': 'Oklahoma City'}))
)

## Attaching twice (one for home and one for away)
games = (games
 .join(teamFouls.rename({'team': 'home_team', 'personal_fouls_per_game':
 'home_team_fouls'}), on = ['season', 'home_team'], how = 'left')
 .join(teamFouls.rename({'team': 'away_team', 'personal_fouls_per_game':
 'away_team_fouls'}), on = ['season', 'away_team'], how = 'left')
)
games

## Expecting that 1,230 games from 2025-26 will have blanks for two columns --
because team fouls file stops at 2024-25
games.filter(pl.col('home_team_fouls').is_null()).group_by('season').len()
```

season	len
str	u32
"2025-2026"	1230

Each row is one game with game info (date, season, teams), crew stats (avg of the 3 referees), and home/away team foul avgs.

For Code Below, We Don't Want Pull Data Every Time - Already Ran It Once

Pulling Each Season's Team Fouls

```
# import time
# from nba_api.stats.endpoints import leaguegameblog

# foullist = []
```

```
# for year in range(2017, 2026):
#     seasonCode = f'{year}-{str(year + 1)[-2:]}' ## nba_api wants '2017-18'
#     not '2017-2018'

#     log = leaguegameLog.LeagueGameLog(
#         season = seasonCode,
#         season_type_all_star= 'Regular Season',
#         timeout = 60,
#     )

#     ## Turn result into polars table
#     result = log.get_dict()['resultSets'][0]
#     seasonLog = pl.DataFrame(result['rowSet'], schema = result['headers'],
# orient='row', infer_schema_length=None)
#     foulList.append(seasonLog.select('GAME_ID', 'TEAM_ABBREVIATION', 'PF',
# 'MIN'))
#     time.sleep(1) ## 1 second pause so we don't get blocked on NBA StopIteration

# ## Stack all seasons and save
# gameLogs = pl.concat(foulList)
# gameLogs.write_csv(here('Data/nba_team_game_logs.csv'))
# gameLogs
```

Reading Commented Out Above and Saving

```
gameLogs = pl.read_csv(here('Data/nba_team_game_logs.csv'))
gameLogs['MIN'].value_counts().sort('MIN')
```

MIN	count
i64	u32
240	20372
265	994
290	120
315	10
340	2

Total Fouls Per Game, Then Join

```
totalFouls = (gameLogs
    .with_columns(game_id = pl.col('GAME_ID').cast(pl.Int64)) ## 0021700001 --
```

```

> 21700001
  .group_by('game_id')
  .agg(
    total_fouls = pl.col('PF').sum(),
    n_teams = pl.len(), ## about 2 every game
    game_minutes = pl.col('MIN').max(),
  )
)

## Attach to games
games = games.join(totalFouls, on = 'game_id', how = 'left')
games

```

game_id	game_date	season	game_code...	away_team	home_team	total_fouls	n_teams	game_minutes
i64	str	str	str	...	f64	i64	u32	i64
22501186	"04/12/2026"	"2025-2026"	"20260412/ ... ORL- BOS"	...	null	42	2	240
22501187	"04/12/2026"	"2025-2026"	"20260412/ ... WAS- CLE"	...	null	39	2	240
22501188	"04/12/2026"	"2025-2026"	"20260412/ ... DETIND"	...	null	34	2	240
...	...	...	...	...	...	...	...	...
21700013	"10/18/2017"	"2017-2018"	"20171018/ ... HOUSAC"	...	19.4	34	2	240
21700001	"10/17/2017"	"2017-2018"	"20171017/ ... BOSCLE"	...	19.9	49	2	240
21700002	"10/17/2017"	"2017-2018"	"20171017/ ... HOUGSW"	...	19.4	41	2	240

Checks

```

## Every game have exactly 2 teams
totalFouls['n_teams'].value_counts()

```

n_teams	count
u32	u32
2	10749

```
## How many games didn't get total_fouls
games['total_fouls'].null_count()
```

45

45 Missing Games (1/2 a Percent)

```
games.filter(pl.col('total_fouls').is_null()).group_by('season').len()
```

season	len
str	u32
"2019-2020"	45

Drop Games, and n_teams Column (Only Needed For One Check)

```
games = (games
          .drop_nulls(subset = 'total_fouls')
          .drop('n_teams')
        )
games
```

game_id	game_date	season	game_code...	home_team	away_team	total_fouls	game_minutes
i64	str	str	str ...	f64	f64	i64	i64
22501186	"04/12/2026"	"2025-2026"	"20260412/ ... ORL- BOS"	null	null	42	240
22501187	"04/12/2026"	"2025-2026"	"20260412/ ... WAS- CLE"	null	null	39	240
22501188	"04/12/2026"	"2025-2026"	"20260412/ ... DETIND"	null	null	34	240
...	...	...	...	...	...	...	...

game_id	game_date	season	game_code...	home_team	away_team	home_fouls	away_fouls	game_minutes
i64	str	str	str	...	f64	f64	i64	i64
21700013	"10/18/2017"	2017-2018	"20171018/ ... HOUSAC"	20.0	19.4	34	240	
21700001	"10/17/2017"	2017-2018	"20171017/ ... BOSCLE"	18.8	19.9	49	240	
21700002	"10/17/2017"	2017-2018	"20171017/ ... HOUGSW"	19.6	19.4	41	240	

The 45 missing games are from 2019-20 (COVID Season) - league shut down

Team Season Avg From gameLogs

```
teamSeasonFouls = (gameLogs
  ## Get season from game ID
  .with_columns(start_year = 2000 + (pl.col('GAME_ID') // 100000) % 100)
  .with_columns(
    season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year')
+ 1).cast(pl.String)
  )
  ## Average fouls per game for each team in season
  .group_by(['season', 'TEAM_ABBREVIATION'])
  .agg(team_fouls = pl.col('PF').mean())
)
teamSeasonFouls
```

season	TEAM_ABBREVIATION	team_fouls
str	str	f64
"2018-2019"	"LAC"	23.329268
"2018-2019"	"LAL"	20.743902
"2019-2020"	"CLE"	18.276923
...	...	...
"2025-2026"	"PHI"	20.329268
"2025-2026"	"NOP"	19.926829
"2025-2026"	"MIL"	19.719512

Swap Out Old Columns For New

```
games = (games
    .drop(['home_team_fouls', 'away_team_fouls']) ## remove old ones (blanks)
    .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'home_team_abbr',
    'team_fouls': 'home_team_fouls'}), on = ['season', 'home_team_abbr'], how =
    'left')
    .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'away_team_abbr',
    'team_fouls': 'away_team_fouls'}), on = ['season', 'away_team_abbr'], how =
    'left')
)
games.select(pl.col('home_team_fouls', 'away_team_fouls').null_count())
```

home_team_fouls	away_team_fouls
u32	u32
0	0

All files combined into one table (games). There's 10,729 games, each with crew stats, team foul avgs, and total_fouls (target number).

Removing Unuseful Columns

```
gamesClean = (games
    .with_columns(game_date = pl.col('game_date').str.to_date('%m/%d/%Y')) ##
    Changing date format
    ## Overtime periods and late season
    .with_columns(
        overtime = ((pl.col('game_minutes') - 240) / 25).cast(pl.Int64),
        days_left = (pl.col('game_date').max().over('season') -
        pl.col('game_date')).dt.total_days(),
    )
    .with_columns(late_season = pl.col('days_left') <= 14)
    .select(
        'game_id', 'game_date', 'season', 'home_team_abbr',
        'away_team_abbr', 'official1', 'official2', 'official3', 'crew_fouls_per_game',
        'crew_experience', 'crew_foul_diff', 'crew_home_win_pct', 'home_team_fouls',
        'away_team_fouls', 'overtime', 'late_season', 'total_fouls' ## Only keeping
        useful columns
    )
    .sort('game_date')
)
gamesClean.write_parquet(here('Data/nba_games_clean.parquet'))
gamesClean
```

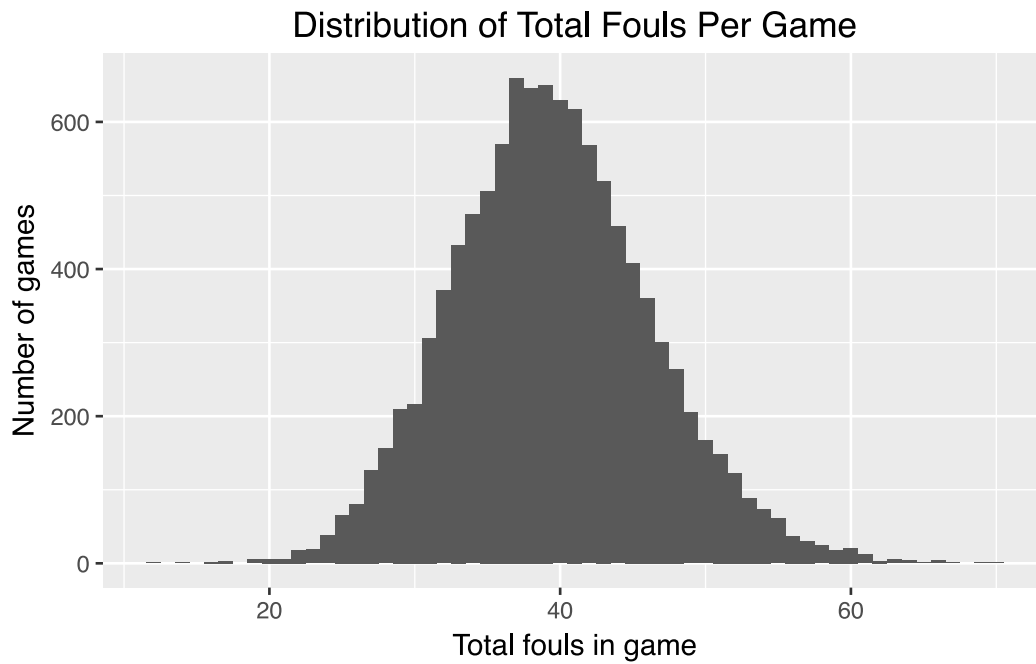
game_id	game_date	season	home_team_abbreviation	away_team_abbreviation	game_time	total_fouls	late_season	total_fouls
i64	date	str	str	...	f64	i64	bool	i64
21700001	2017-10-17	"2017-2018"	"CLE"	...	19.731707	0	false	49
21700002	2017-10-17	"2017-2018"	"GSW"	...	19.47561	0	false	41
21700003	2017-10-18	"2017-2018"	"DET"	...	17.182927	0	false	36
...	...	...	...	...	...	...	...	...
22501198	2026-04-12	"2025-2026"	"LAL"	...	21.134146	0	true	34
22501199	2026-04-12	"2025-2026"	"LAC"	...	19.609756	0	true	45
22501200	2026-04-12	"2025-2026"	"POR"	...	19.682927	0	true	39

EDA

Histogram of Total Fouls

```
from plotnine import *
import pyarrow

(ggplot(gamesClean, aes(x = 'total_fouls'))
 + geom_histogram(binwidth=1)
 + labs(x = 'Total fouls in game', y = 'Number of games', title = 'Distribution
of Total Fouls Per Game')
)
```



Look at Unusual Games

```
gamesClean['total_fouls'].describe()
```

statistic	value
str	f64
"count"	10729.0
"null_count"	0.0
"mean"	39.452698
...	...
"50%"	39.0
"75%"	44.0
"max"	70.0

Games With Fewest and Most Fouls

```
## Lowest
gamesClean.sort('total_fouls').select('game_date', 'home_team_abbr',
'away_team_abbr', 'total_fouls').head(10)
```

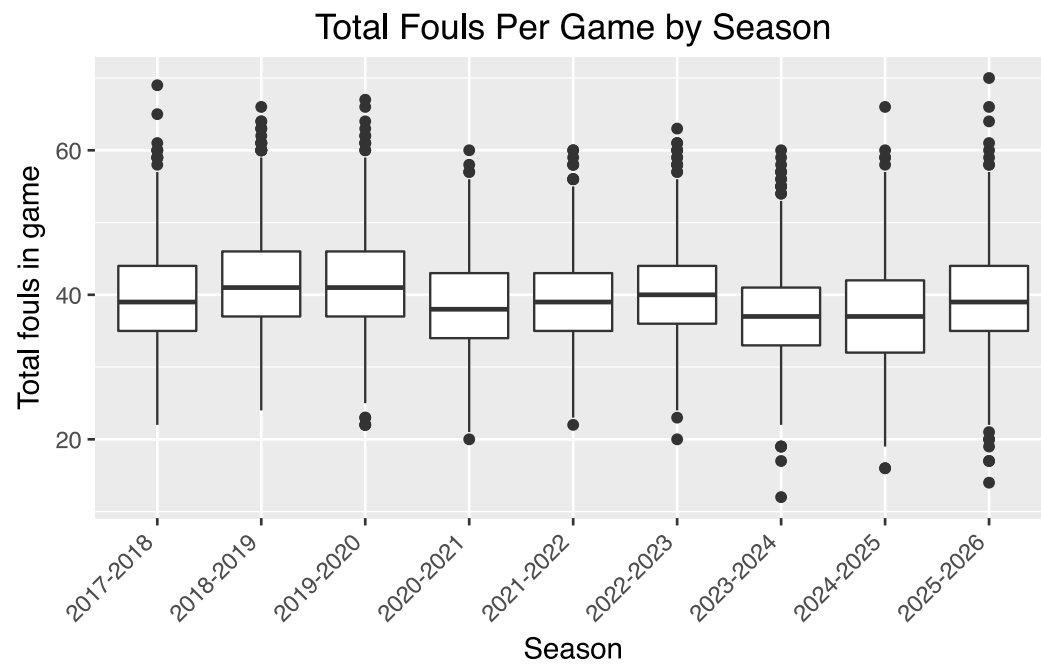
game_date	home_team_abbr	away_team_abbr	total_fouls
date	str	str	i64
2024-04-09	"MIL"	"BOS"	12
2026-01-12	"IND"	"BOS"	14
2025-04-11	"PHX"	"SAS"	16
...	...	...	...
2024-04-09	"UTA"	"DEN"	19
2024-04-12	"DAL"	"DET"	19
2024-04-14	"CLE"	"CHA"	19

```
## Highest
gamesClean.sort('total_fouls', descending=True).select('game_date',
'home_team_abbr', 'away_team_abbr', 'total_fouls').head(10)
```

game_date	home_team_abbr	away_team_abbr	total_fouls
date	str	str	i64
2025-10-23	"IND"	"OKC"	70
2017-11-29	"PHI"	"WAS"	69
2020-07-31	"DAL"	"HOU"	67
...	...	...	...
2017-12-31	"HOU"	"LAL"	65
2019-01-02	"PHX"	"PHI"	64
2019-03-01	"ATL"	"CHI"	64

Fouls Over Time

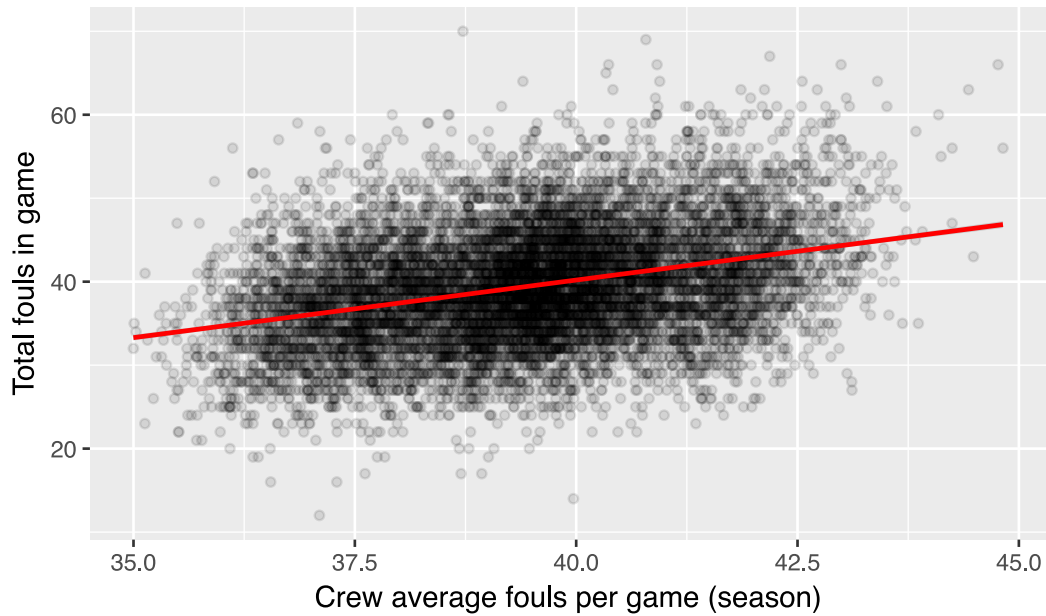
```
(ggplot(gamesClean, aes(x = 'season', y = 'total_fouls'))
+ geom_boxplot()
+ labs(x = 'Season', y = 'Total fouls in game', title = 'Total Fouls Per
Game by Season')
+ theme(axis_text_x = element_text(rotation = 45, ha = 'right'))
)
```



Does The Crew Actually Matter

```
(ggplot(gamesClean, aes(x = 'crew_fouls_per_game', y = 'total_fouls'))  
  + geom_point(alpha = 0.1)  
  + geom_smooth(method = 'lm', color = 'red')  
  + labs(x = 'Crew average fouls per game (season)', y = 'Total fouls in game',  
    title = 'Crew Foul Tendency vs. Total Fouls')  
)
```

Crew Foul Tendency vs. Total Fouls



Correlations

```
gamesClean.select(
  'total_fouls', 'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff',
  'crew_home_win_pct', 'home_team_fouls', 'away_team_fouls', 'overtime'
).corr()
```

total_fouls	crew_fouls_per_game	crew_experience	crew_foul_diff	home_team_fouls	away_team_fouls	overtime
f64	f64	f64	f64	f64	f64	f64
1.0	0.336008	-0.043219	0.045756	0.053492	0.295869	0.288569
0.336008	1.0	-0.117469	0.194949	0.241477	0.480797	0.478423
-0.043219	-0.117469	1.0	0.039604	0.011987	-0.063816	-0.067382
...	...	...	...	...	...	...
0.295869	0.480797	-0.063816	0.066803	0.115679	1.0	0.245479
0.288569	0.478423	-0.067382	0.14622	0.154994	0.245479	1.0
0.209044	0.044965	-0.019305	-0.008561	0.004029	0.015176	0.001517